Core 90 - Hash maps and sets

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	4	2
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Hash maps and sets | 7 questions | tap a title to jump to it

E	1. Robot Status Message Deduplication Window	1Point3Acres	2026-09-10
M	2. Connected Crop Arrangement on a Grid	1Point3Acres	2026-10-04
M	3. Filter Repeated Robot Status Messages	PracHub	2026-07-05
M	4. Next-Word Predictor	1Point3Acres	2026-09-02
M	5. Solve array-sum and city-community problems	PracHub	2026-02-20
H	6. Maximum-Area Rectangle from Planar Points	1Point3Acres	2026-09-29
H	7. Smallest Missing Positive Integer in Linear Time and Constant Extra Space	PracHub	2026-09-09

STUDY PATH: question 9 of 167 (Core 90)

Priority score 6.35 | Hash maps and sets | asked 40 days ago | frequency Low

EASY | 1Point3Acres | onsite-coding | 1 of 7

Robot Status Message Deduplication Window

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 51.2 30 DAYS 33 PRIORITY 34/228

ABOUT THIS QUESTION

Topic: Core 90 - Hash maps and sets

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (related)

Frequency: Low | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

QUESTION

Process timestamped robot messages and display a message only when it has not been displayed within the prior 10 seconds. Hidden arrivals do not refresh the last-displayed timestamp, and the exact 10-second boundary must be clarified.

Process a stream of (time, message) status events from a robot.
If the same message was displayed within the past 10 seconds, hide the new event.
Otherwise display it and update that message's last-displayed time.
Track the last time each message was displayed, not the last time it was received. A hidden duplicate must not refresh the window.
Clarify whether an event exactly 10 seconds after the prior display is allowed: the comparison may be > 10 or >= 10.

Logger API variant

Implement shouldPrintMessage(timestamp, message), returning true when the message has not been printed in the past 10 seconds and false otherwise.
Handle timestamps arriving out of order, including timestamp 12 arriving before timestamp 10.

EXAMPLES

```
time=10, message="solar panel activated" -> show
time=13, message="solar panel activated" -> hide
time=21, message="solar panel activated" -> show
```

The event at 21 compares against the display at 10, not the hidden arrival at 13.

APPROACH

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

HOW TO THINK ABOUT IT

The state that matters is the last time each message was displayed, not the last time it arrived. A message that is hidden must not move that time, otherwise a message repeated every few seconds would be hidden forever. A hash map from message to last displayed time answers each event in O(1).

Basis: LeetCode 359, Logger Rate Limiter (a message may be shown again only after the window has passed).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
shown = []  # (time, message) displayed
def on_event(t, msg):
    for t0, m0 in shown:
        if m0 == msg and t - t0 < 10:
            return False  # hidden
    shown.append((t, msg))
    return True  # displayed
```

TIME	SPACE
O(displayed) per event	O(displayed)

WHY IT IS SLOW

Each event scans the whole history. A map from message to its last displayed time answers each event in O(1).

OPTIMAL SOLUTION · Hash map of last displayed time

TIME	SPACE
O(1) per event	O(distinct messages)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Events can arrive out of order.

Decide the contract first: either a late event is judged against the stored time with abs(t - last), or you keep a sorted history per message and compare with the nearest displayed neighbours on both sides. The simple map is only correct for a chronological stream.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Robot Status Message Deduplication Window
#
# Process timestamped robot messages and display a message
# only when it has not been displayed within the prior 10
# seconds. Hidden arrivals do not refresh the last-
# displayed timestamp, and the exact 10-second boundary
# must be clarified. Process a stream of (time, message)
# status events from a robot. If the same message was
# displayed within the past 10 seconds, hide the new
# event. Otherwise display it and update that message's
# last-displayed time. Track the last time each message
# was displayed, not the last time it was received. A
# hidden duplicate must not refresh the window. Clarify
# whether an event exactly 10 seconds after the prior
# display is allowed: the comparison may be > 10 or >= 10.
# Logger API variant Implement
# shouldPrintMessage(timestamp, message), returning true
# when the message has not been printed in the past 10
# seconds and false otherwise.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Events can arrive out of order.
#
# INTERVIEW NOTES
# The state that matters is the last time each message was
# displayed, not the last time it arrived. A message that
# is hidden must not move that time, otherwise a message
# repeated every few seconds would be hidden forever. A
# hash map from message to last displayed time answers
# each event in O(1).
#
# BRUTE FORCE (say it first): Keep every displayed time
# and scan them
# Store the list of every (time, message) shown so far and
# scan it for the same message within the window. Correct,
# but the scan grows with the whole history. Time
# O(displayed) per event. Space O(displayed).


from bisect import bisect_left, bisect_right, insort


# BRUTE FORCE (say it first): Keep every displayed time
# and scan them | time O(displayed) per event
class DedupBrute:
    def __init__(self, window=10):
        self.window, self.shown = window, []

    def show(self, t, msg):
        if any(
            m == msg and t - s < self.window
            for s, m in self.shown
        ):
            return False
        self.shown.append((t, msg))
        return True


# BASE QUESTION (optimal): Hash map of last displayed time
# Idea: If the message was displayed less than `window`
# seconds ago hide it without changing the stored time;
# otherwise display it and store the new time.
class Dedup:
    """Chronological stream: remember the last display."""

    def __init__(self, window=10, boundary_allowed=True):
        self.window, self.last = window, {}
        # True: exactly `window` seconds later may show
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        # is this gap still inside the hiding window?
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        # only a displayed event starts a new window
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True


# FOLLOW-UP 1: Events can arrive out of order.
# Answer: Decide the contract first: either a late event
# is judged against the stored time with abs(t - last), or
# you keep a sorted history per message and compare with
# the nearest displayed neighbours on both sides.
class DedupOutOfOrder:
    """Events may arrive late: look at both neighbours."""

    def __init__(self, window=10):
        self.window = window
        # message -> sorted list of displayed times
        self.shown = {}

    def show(self, t, msg):
        times = self.shown.setdefault(msg, [])
        # any display within `window` on either side?
        lo = bisect_right(times, t - self.window)
        hi = bisect_left(times, t + self.window)
        if lo < hi:
            return False
        insort(times, t)
        return True


if __name__ == "__main__":
    d = Dedup()
    # 13 is hidden and does not move the window
    assert [d.show(t, "solar") for t in (10, 13, 21)] == [
        True, False, True]
    # exactly 10 seconds later shows by default
    d = Dedup()
    d.show(0, "a")
    assert d.show(10, "a")
    # ... but is hidden if the boundary is not allowed
    d = Dedup(boundary_allowed=False)
    d.show(0, "a")
    assert not d.show(10, "a")
    o = DedupOutOfOrder()
    assert o.show(20, "x")
    # a late event close to a displayed one is hidden
    assert not o.show(15, "x")
    # a late event far enough from every display shows
    assert o.show(5, "x")
```

NOTES FROM CANDIDATES

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.
Resolve the exact-boundary rule before coding and make the comparison match that contract.

For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

PREPARATION

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

STUDY PATH: question 10 of 167 (Core 90)

Priority score 5.5 | Hash maps and sets | asked 16 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 2 of 7

Connected Crop Arrangement on a Grid

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 207/228**

ABOUT THIS QUESTION

Topic: Core 90 - Hash maps and sets

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/connected-crop-arrangement-grid

QUESTION

Given an N x M grid and K crop counts summing to N*M, fill every cell so each crop's cells form one 4-connected region; return any valid arrangement. Follow-up: two adjacent square gardens (N x N and M x M) with counts summing to N^2 + M^2.

A garden is an N x M grid. There are K crop types labeled 1..K; crop i must occupy exactly count[i] cells.
Guaranteed: sum(count) = N * M, so every cell is used.
For every crop type, all of its cells must form one connected component under 4-directional adjacency (up, down, left, right; diagonals do not count).
Different crops have no relationship constraints with each other. Return any valid arrangement.

Follow-up: two square gardens

Instead of one rectangle, there are two square gardens placed directly next to each other: garden A is N x N, garden B is M x M; together they form the planting area.
Now sum(count) = N^2 + M^2. The same single-connected-region rule applies across the combined shape.

EXAMPLES

Follow-up shapes for N = 2, M = 3 (digits mark garden A = 1, garden B = 2):

```
11222
11222
  222
```

or

```
11
11
222
222
222
```

APPROACH

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

HOW TO THINK ABOUT IT

Walk the grid in a snake (left to right on one row, right to left on the next) so that every cell is next to the one before it. If you give crop 1 the first count[1] cells of that walk, crop 2 the next count[2] cells and so on, each crop is a contiguous stretch of the walk, so its cells are connected. No search or backtracking is needed.

Basis: Written for this question: a constructive fill along a Hamiltonian path of the grid (a snake), no LeetCode problem matches it exactly.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fill(crop):
    if crop == len(counts):
        return True
    size = counts[crop]
    for cells in combinations(free_cells, size):
        if is_connected(cells):
            place(cells, crop)
            if fill(crop + 1):
                return True
            remove(cells)
    return False
```

TIME	SPACE
exponential	O(n * m)

WHY IT IS SLOW

Exponential in the number of cells. Filling the crops along a snake path through the grid is O(n * m) and every crop stays connected by construction.

OPTIMAL SOLUTION · Fill crops along a snake path

TIME	SPACE
O(n * m)	O(n * m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two adjacent square gardens (N x N next to M x M).

Use a different Hamiltonian path of the combined L-shaped area: snake down and up the columns of the first garden so that the last column ends in the top row, step into the second garden there and snake through it. Hand out the crop counts along that path exactly as before. The code checks that this path visits every cell once with each step adjacent, for every size from 1 to 6.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Connected Crop Arrangement on a Grid
#
# Given an N x M grid and K crop counts summing to N*M,
# fill every cell so each crop's cells form one
# 4-connected region; return any valid arrangement.
# Follow-up: two adjacent square gardens (N x N and M x M)
# with counts summing to N^2 + M^2. A garden is an N x M
# grid. There are K crop types labeled 1..K; crop i must
# occupy exactly count[i] cells. Guaranteed: sum(count) =
# N * M, so every cell is used. For every crop type, all
# of its cells must form one connected component under
# 4-directional adjacency (up, down, left, right;
# diagonals do not count). Different crops have no
# relationship constraints with each other. Return any
# valid arrangement. Follow-up: two square gardens Instead
# of one rectangle, there are two square gardens placed
# directly next to each other: garden A is N x N, garden B
# is M x M; together they form the planting area. Now
# sum(count) = N^2 + M^2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two adjacent square gardens (N x N next to M x M).
#
# INTERVIEW NOTES
# Walk the grid in a snake (left to right on one row,
# right to left on the next) so that every cell is next to
# the one before it. If you give crop 1 the first count[1]
# cells of that walk, crop 2 the next count[2] cells and
# so on, each crop is a contiguous stretch of the walk, so
# its cells are connected. No search or backtracking is
# needed.
#
# BRUTE FORCE (say it first): Backtrack over every
# assignment
# Try every way to place the counts and test connectivity.
# Correct and only possible on tiny grids, which makes it
# a good oracle for the constructive answer. Time
# exponential. Space O(n * m).


import itertools
from collections import defaultdict


# BRUTE FORCE (say it first): Backtrack over every
```

```
# assignment | time exponential
def crops_brute_possible(n, m, counts):
    cells = [(i, j) for i in range(n) for j in range(m)]

    def connected(group):
        group = set(group)
        if not group:
            return True
        stack, seen = [next(iter(group))], set()
        while stack:
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack += [
                x
                for x in (
                    (r + 1, c),
                    (r - 1, c),
                    (r, c + 1),
                    (r, c - 1),
                )
                if x in group
            ]
        return seen == group

    def go(idx, remaining):
        if idx == len(counts):
            return True
        for pick in itertools.combinations(
            remaining, counts[idx]
        ):
            if connected(pick) and go(
                idx + 1,
                [c for c in remaining if c not in pick],
            ):
                return True
        return False

    return go(0, cells)


# HELPERS for the base question
def snake_cells(n, m):
    # row 0 left to right, row 1 right to left, ...
    return [(i, j if i % 2 == 0 else m - 1 - j)
            for i in range(n) for j in range(m)]


def fill_along(grid, path, counts):
    # hand out the path to crop 1, crop 2, ... in turn
    k = 0
    for crop, cnt in enumerate(counts, 1):
        for _ in range(cnt):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid


# HELPERS for follow-up 1
def two_gardens_path(n, m):
    # garden A is n x n; garden B is m x m on its right
    path = []
    # snake down and up the columns of garden A
    for c in range(n):
        # the last column must end in row 0
        down = (c % 2 == 0) == (n % 2 == 0)
        rows = range(n)
        if not down:
            rows = range(n - 1, -1, -1)
        path += [(r, c) for r in rows]
    # step into garden B at row 0 and snake through it
    for r in range(m):
        cols = range(n, n + m) if r % 2 == 0 \
            else range(n + m - 1, n - 1, -1)
        path += [(r, c) for c in cols]
    return path
```

```python
# BASE QUESTION (optimal): Fill crops along a snake path
# Idea: Generate the snake order of the cells and hand out
# count[0] cells to crop 1, the next count[1] to crop 2,
# and so on. Consecutive cells of the snake share an edge,
# so each crop is connected.
def arrange_crops(n, m, counts):
    # every cell is used exactly once
    assert sum(counts) == n * m
    grid = [[0] * m for _ in range(n)]
    # touching path cells keep each crop connected
    return fill_along(grid, snake_cells(n, m), counts)


# FOLLOW-UP 1: Two adjacent square gardens (N x N next to
# M x M).
# Answer: Use a different Hamiltonian path of the combined
# L-shaped area: snake down and up the columns of the
# first garden so that the last column ends in the top
# row, step into the second garden there and snake through
# it.
def arrange_two_gardens(n, m, counts):
    assert sum(counts) == n * n + m * m
    # the grid is wide enough for both gardens
    grid = [[0] * (n + m) for _ in range(max(n, m))]
    path = two_gardens_path(n, m)
    return fill_along(grid, path, counts)


# HELPERS for the checks below
def crops_valid(grid, counts):
    cells = defaultdict(set)
    for i, row in enumerate(grid):
        for j, v in enumerate(row):
            if v:
                cells[v].add((i, j))
    for crop, cnt in enumerate(counts, 1):
        group = cells[crop]
        # the right number of cells
        if len(group) != cnt:
            return False
        # flood fill from one cell must reach them all
        stack, seen = [next(iter(group))], set()
        while stack:
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            for x in ((r + 1, c), (r - 1, c),
                      (r, c + 1), (r, c - 1)):
                if x in group:
                    stack.append(x)
        if seen != group:
            return False
    return True
```

```python
if __name__ == "__main__":
    # a 2 x 3 field with two crops
    g = arrange_crops(2, 3, [2, 4])
    assert crops_valid(g, [2, 4])
    # many tiny crops still connect
    ones = [1] * 9
    assert crops_valid(arrange_crops(3, 3, ones), ones)
    # two gardens: 2 x 2 next to 3 x 3 (13 cells)
    counts = [5, 4, 4]
    g2 = arrange_two_gardens(2, 3, counts)
    assert crops_valid(g2, counts)
    # the path visits every cell once, step by step
    p = two_gardens_path(2, 3)
    assert len(set(p)) == 13
    assert all(abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1
               for a, b in zip(p, p[1:]))
```

NOTES FROM CANDIDATES

Asked as the first-round online coding interview of an early-career L3 loop.
Because any valid arrangement is accepted, look for a constructive traversal order that keeps consecutive cells adjacent, rather than backtracking.
For the follow-up, the two gardens can be stacked side by side or one above the other; clarify how they are joined (which edges touch and the offset) before designing the traversal.

PREPARATION

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

STUDY PATH: question 11 of 167 (Core 90)

Priority score 4.55 | Hash maps and sets | asked 107 days ago

MEDIUM | PracHub | Technical Screen | 3 of 7

★ TOP 75: rank 51 in the Medium and Hard priority list

★ Filter Repeated Robot Status Messages

POSTED 2026-07-05 | 107 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 33** | **PRIORITY 32/228**

ABOUT THIS QUESTION

Topic: Core 90 - Hash maps and sets

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (inferred from overview)

Posted on PracHub: 2026-07-05 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/filter-repeated-robot-status-messages

QUESTION

Overview: Filter a timestamped robot status stream so a message reappears only when strictly more than ten seconds have passed since its last displayed occurrence. Preserve input order, track each exact case-sensitive message independently, and ensure hidden repeats do not reset the interval.

APPROACH

Hash map message -> last displayed timestamp; display only if now - last > 10 (strictly), then update. Preserve input order. O(1) per message.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact rule: show a message again only if more than 10 seconds have passed since it was last displayed (strictly more), whether messages are compared case-sensitively (yes), and whether hidden repeats extend the interval (no).

BRUTE FORCE TO SAY OUT LOUD

For each message scan the already displayed ones for the last occurrence. O(n^2).

OPTIMAL IDEA

Keep a dict from message to the time it was last displayed. For each message in order: if it is not in the dict or now - last > 10, display it and set last = now; otherwise skip it and do not update last. Output preserves input order. O(1) per message.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

A repeat at exactly 10 seconds (hidden), at 11 seconds (shown), a hidden repeat followed by another repeat 10 seconds after the first display (shown, since hidden repeats do not reset), different casing (different messages), the first message.

LIKELY FOLLOW-UPS

Bounded memory (evict messages older than the interval), or a per-robot map.

PITFALLS

Updating the last time on a hidden message. Using at least instead of strictly more than.

HOW TO THINK ABOUT IT

The decision for a message depends only on when that exact message was last displayed, so a dictionary from message to last display time is all the state you need. The detail that decides correctness: the stored time changes only when a message is shown (a hidden repeat must not reset the clock), and the comparison is strict (> 10).

Basis: Written (compare LeetCode 359, Logger Rate Limiter). Rule: show a message again only when strictly more than 10 seconds have passed since it was last shown; hidden repeats do not extend the interval; messages are case-sensitive; input order is kept.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
shown = []  # (time, message)
def on_message(t, msg):
    last = None
    for t0, m0 in shown:
        if m0 == msg:
            last = t0  # most recent display
    if last is None or t - last > 10:
        shown.append((t, msg))
        return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each message searches the whole history, O(n^2). A dictionary from message to its last displayed time makes each decision O(1).

OPTIMAL SOLUTION · Dictionary from message to the time it was last shown

TIME	SPACE
O(n)	O(distinct messages)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The stream never ends; memory must be bounded.

A message older than the gap can never be blocked again, so evict it. Keep a min-heap of (time shown, message) and pop entries older than the gap before each decision; delete the dictionary key only if it still points at that time.

FOLLOW-UP 2 Several robots send messages.

Use (robot id, message) as the key, or one filter per robot. The same logic applies per key.

FOLLOW-UP 3 The interval depends on the message severity.

Look up the gap from a severity table and use it in the comparison; the state (last shown time) is unchanged.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Filter Repeated Robot Status Messages
#
# Filter a timestamped robot status stream so a message
# reappears only when strictly more than ten seconds have
# passed since its last displayed occurrence. Preserve
# input order, track each exact case-sensitive message
# independently, and ensure hidden repeats do not reset
# the interval.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The stream never ends; memory must be bounded.
# 2. Several robots send messages.
# 3. The interval depends on the message severity.
#
# INTERVIEW NOTES
# The decision for a message depends only on when that
# exact message was last displayed, so a dictionary from
# message to last display time is all the state you need.
# The detail that decides correctness: the stored time
# changes only when a message is shown (a hidden repeat
# must not reset the clock), and the comparison is strict
# (> 10).
#
# BRUTE FORCE (say it first): Scan the already shown
# messages for each new one
# Keep the list of shown (time, message) pairs and, for
# each new message, look backward for its last shown time.
# Correct and the oracle; each message searches the whole
# history. Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Scan the already shown
# messages for each new one | time O(n^2)
def filter_status_brute(stream):
    shown, out = [], []
    for ts, msg in stream:
        last = next(
            (t for t, m in reversed(shown) if m == msg),
            None,
        )
        if last is None or ts - last > 10:
            shown.append((ts, msg))
            out.append((ts, msg))
    return out


# BASE QUESTION (optimal): Dictionary from message to the
# time it was last shown
# Idea: For each message in order: if it was never shown
# or now - last > 10, emit it and set last = now;
# otherwise drop it and leave last unchanged. One
# dictionary lookup per message.
def filter_status(stream, gap=10):
    # show a message only if MORE than `gap` seconds
    # passed
    # since it was last shown; hidden repeats change
    # nothing
    last, shown = {}, []
    for ts, msg in stream:
        if msg not in last or ts - last[msg] > gap:
            last[msg] = ts
            shown.append((ts, msg))
    return shown


# FOLLOW-UP 1: The stream never ends; memory must be
# bounded.
# Answer: A message older than the gap can never be
# blocked again, so evict it. Keep a min-heap of (time
# shown, message) and pop entries older than the gap
# before each decision; delete the dictionary key only if
# it still points at that time.
class BoundedFilter:
    """An endless stream: forget messages that can no
    longer be blocked, so memory stays small."""

    def __init__(self, gap=10):
        self.gap = gap
        self.last = {}
        # (time shown, message), oldest first
        self.heap = []

    def show(self, ts, msg):
        # drop entries older than the gap
        while self.heap:
            if ts - self.heap[0][0] <= self.gap:
                break
            old_ts, old_msg = heapq.heappop(self.heap)
            # delete only if it was not shown again since
            if self.last.get(old_msg) == old_ts:
                del self.last[old_msg]
        if msg in self.last:
            return False
        self.last[msg] = ts
        heapq.heappush(self.heap, (ts, msg))
        return True


# FOLLOW-UP 2: Several robots send messages.
# Answer: Use (robot id, message) as the key, or one
# filter per robot. The same logic applies per key.
class PerRobotFilter:
    """Several robots; the gap depends on severity."""

    def __init__(self, gaps, default=10):
        self.gaps, self.default = gaps, default
        self.last = {}

    def show(self, robot, ts, msg, severity=None):
        key = (robot, msg)
        gap = self.gaps.get(severity, self.default)
```

```
        last = self.last.get(key)
        if last is not None and ts - last <= gap:
            return False
        self.last[key] = ts
        return True


if __name__ == "__main__":
    stream = [(0, "a"), (5, "a"), (10, "a"), (11, "a"),
              (12, "b"), (22, "a")]
    # 10 is hidden (not MORE than 10 after 0); 11 shows
    assert filter_status(stream) == [(0, "a"), (11, "a"),
                                     (12, "b"), (22, "a")]
    # messages are case sensitive
    assert filter_status([(0, "A"), (1, "a")]) == [
        (0, "A"), (1, "a")]
    b = BoundedFilter()
    shows = [b.show(ts, msg) for ts, msg in stream]
    assert shows == [True, False, False, True, True, True]
    assert len(b.last) <= 2
    r = PerRobotFilter({"high": 2})
```

```
    assert r.show("r1", 0, "x")
    assert not r.show("r1", 9, "x")
    # another robot, and a stricter severity
    assert r.show("r2", 1, "x")
    assert r.show("r1", 1, "y", "high")
    assert r.show("r1", 4, "y", "high")
```

FUNCTION AND TEST CASES

Signature

```
def filter_messages(events: list[tuple[int, str]]) -> list
[tuple[int, str]]:
```

Sample call

```
filter_messages([])
```

Test cases

Input	Expected
`([],)`	`[]`
`([[10, "solar panel activated"], [13, "solar panel activated"], [20, "solar panel activated"], [21, "solar panel activated"]],)`	`[[10, "solar panel activated"], [21, "solar panel activated"]]`

ALSO REPORTED · 1 more

Robot Status Message Deduplication Window | 1Point3Acres | onsite-coding | asked 2026-09-10 | Low

https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

ALSO REPORTED · 1 more

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.
Resolve the exact-boundary rule before coding and make the comparison match that contract.
For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

Preparation: Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand. Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

STUDY PATH: question 12 of 167 (Core 90)

Priority score 4.5 | Hash maps and sets | asked 48 days ago | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 4 of 7

Next-Word Predictor

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 224/228**

ABOUT THIS QUESTION

Topic: Core 90 - Hash maps and sets

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/next-word-frequency-predictor

QUESTION

Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.SWEstringhashmapdata-structureFrequencySingle reportLast asked2026-09-02Stageonsite-coding

APPROACH

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how sentences are split and cased, how ties are broken, what to return for an unseen word, and whether training is incremental.

BRUTE FORCE TO SAY OUT LOUD

Store all sentences and scan them for each prediction. O(corpus) per query.

OPTIMAL IDEA

Keep a dict from a word to a dict of next-word counts, and a separate best[word] updated whenever a count changes (if the new count beats the current best, or ties and wins the tie-break). Prediction is a dict lookup, O(1). Training is O(total words).

DRY-RUN THESE CASES

A repeated pair, a tie, a new leader taking over, a word with no successor, the last word of a sentence.

LIKELY FOLLOW-UPS

Top-k predictions (keep a heap per word), bigram versus trigram context, or decay for recent data.

INTERVIEW-DAY PLAN

PITFALLS

Updating best only on strict increase and ignoring the tie rule. Counting across sentence boundaries.

HOW TO THINK ABOUT IT

Prediction in O(1) means the answer must already be sitting in a table when the query arrives, so do the work while training. Keep counts per (word, next word) and, for each word, the current best successor. Each count only increases by one, so the best can be updated by comparing just the one count that changed.

Basis: Written; a bigram language model. Assumed rules (confirm them): words are split on spaces and compared case-sensitively; pairs never cross sentence boundaries; ties go to the alphabetically smaller word; an unseen word predicts None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
sentences = []
def train(sentence):
    sentences.append(sentence.split())
def predict(word):
    counts = Counter()
    for words in sentences:
        for a, b in zip(words, words[1:]):
            if a == word:
                counts[b] += 1
    if not counts:
        return None
    return max(counts, key=counts.get)
```

TIME	SPACE
O(corpus size) per prediction	**O(corpus size)**

WHY IT IS SLOW

Each prediction is a full pass over the corpus. Counting pairs while training and keeping the best successor per word makes prediction O(1).

OPTIMAL SOLUTION · Counts plus a maintained best successor

TIME	SPACE
O(1) per pair when training and per prediction	**O(number of distinct pairs)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the top K predictions.

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

FOLLOW-UP 2 Use two words of context (trigrams).

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

FOLLOW-UP 3 The corpus keeps changing and old text should fade.

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Next-Word Predictor
#
# Process training data and predict the most frequent next
# word for a supplied word. Maintain prediction state
# during processing so prediction can run in constant
# time.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the top K predictions.
# 2. Use two words of context (trigrams).
# 3. The corpus keeps changing and old text should fade.
#
# INTERVIEW NOTES
# Prediction in O(1) means the answer must already be
# sitting in a table when the query arrives, so do the
# work while training. Keep counts per (word, next word)
# and, for each word, the current best successor. Each
# count only increases by one, so the best can be updated
# by comparing just the one count that changed.
#
# BRUTE FORCE (say it first): Scan the whole corpus on
# every prediction
# Keep the sentences and count successors of the query
# word from scratch on each call. Simple and correct, but
# every query is as slow as one full pass over the data.
# Time O(corpus size) per prediction. Space O(corpus
# size).


import heapq
from collections import Counter, defaultdict


# BRUTE FORCE (say it first): Scan the whole corpus on
# every prediction | time O(corpus size) per prediction
class PredictorBrute:
    def __init__(self):
        self.sentences = []

    def train(self, sentence):
        self.sentences.append(sentence.split())

    def predict(self, word):
        c = Counter(
            s[i + 1]
            for s in self.sentences
            for i in range(len(s) - 1)
            if s[i] == word
        )
        return (
            min(c, key=lambda w: (-c[w], w))
            if c
            else None
        )


# BASE QUESTION (optimal): Counts plus a maintained best
# successor
# Idea: counts[word][next] is incremented for every
# adjacent pair. After the increment compare that one
# count with the current best for the word (higher count
# wins, then the smaller word).
# FOLLOW-UP 1: Return the top K predictions.
# Answer: Keep the Counter per word and use
# heapq.nsmallest on (-count, word): O(d log K) per query,
# or keep a sorted structure per word if queries dominate.
# FOLLOW-UP 2: Use two words of context (trigrams).
# Answer: The key becomes the pair (previous two words)
# and everything else is identical, at the price of more
# memory. Back off to the bigram table when the pair was
# never seen.
class Predictor:
    # FOLLOW-UP 2: orders=(2, 1) means two words of
    # context
    def __init__(self, orders=(2, 1)):
        # orders: how many previous words are the context
        self.orders = sorted(orders, reverse=True)
        # order -> context tuple -> Counter of next words
        self.counts = {k: defaultdict(Counter)
                       for k in orders}
        # order -> context tuple -> best next word so far
        self.best = {k: {} for k in orders}

    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                ctx = tuple(words[i:i + k])
                nxt = words[i + k]
                self.update(k, ctx, nxt)

    def update(self, k, ctx, nxt):
        table = self.counts[k][ctx]
        table[nxt] += 1
        cur = self.best[k].get(ctx)
        # more counts win; a tie goes to the smaller word
        better = cur is None or table[nxt] > table[cur]
        tie = cur is not None and table[nxt] == table[cur]
        if better or (tie and nxt < cur):
            self.best[k][ctx] = nxt

    def predict(self, *context):
        # O(1): try the longest context, then back off
        for k in self.orders:
            if len(context) >= k:
                ctx = tuple(context[len(context) - k:])
                if ctx in self.best[k]:
                    return self.best[k][ctx]
        return None

    # FOLLOW-UP 1: the top K predictions
    def top_k(self, word, k):
        # k most likely next words after one word
```

```
        by_ctx = self.counts.get(1, {})
        table = by_ctx.get((word,), Counter())
        return heapq.nsmallest(
            k, table, key=lambda w: (-table[w], w))


if __name__ == "__main__":
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    # "love" is followed by cats twice, dogs once
    assert p.predict("love") == "cats"
    # two words of context are tried first
    assert p.predict("i", "love") == "cats"
    # an unseen pair backs off to the last word
    assert p.predict("we", "love") == "cats"
    assert p.predict("zebra") is None
    assert p.top_k("love", 2) == ["cats", "dogs"]
    # a tie goes to the alphabetically smaller word
    q = Predictor()
    q.train("a z")
```

```
    q.train("a b")
    assert q.predict("a") == "b"
```

NOTES FROM CANDIDATES

Exact method signatures, tokenization rules, frequency ties, and behavior for an unseen word are unspecified.
The candidate used a dictionary and stored the current best successor during processing. Calling this state a cache caused a misunderstanding: the interviewer initially thought updates would happen during prediction. Explain the update timing explicitly.
After agreeing on tokenization and sequence boundaries, increment a counter for each adjacent pair (previous_word, next_word). For each previous word, retain a successor whose count is maximal. Compare the updated successor with the retained best immediately during processing; prediction only reads that stored best.
With insertion-only counts, a pair whose count did not change cannot newly overtake the best. Under a practice convention that retains the incumbent on ties, replace the best only when the updated count is strictly larger. If a different tie rule is required, apply it explicitly on equal counts.
Hash-table updates per observed pair and prediction are expected O(1), excluding tokenization/string hashing costs. Store O(P + V) state for P distinct pairs and V predecessor words. Clarify unseen-word results and whether separate training sequences create cross-boundary pairs.

PREPARATION

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

STUDY PATH: question 13 of 167 (Core 90)

Priority score 3.64 | Hash maps and sets | asked 242 days ago

MEDIUM | PracHub | Technical Screen | 5 of 7

★ TOP 75: rank 13 in the Medium and Hard priority list

★ Solve array-sum and city-community problems

POSTED 2026-02-20 | 242 DAYS AGO | OLDER

STUDY: TYPE | **ALL TIME 66** | **30 DAYS 46.9** | **PRIORITY 7/228**

ABOUT THIS QUESTION

Topic: Core 90 - Hash maps and sets

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #560 Subarray Sum Equals K, LeetCode #200 Number of Islands

LeetCode match: LC 560 Subarray Sum Equals K + LC 200 Number of Islands (inferred from overview)

Posted on PracHub: 2026-02-20 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/solve-array-sum-and-city-community-problems

QUESTION

Overview: This question evaluates array-processing and grid/graph connectivity skills, covering counting subarrays with a target sum and identifying connected building communities in a grid; it belongs to the Coding & Algorithms domain and involves streaming/online algorithm reasoning as well as grid traversal concepts.

APPROACH

Prefix sum + hash map of counts for the subarray target; communities = connected components via DFS/union-find on the grid.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Array: count subarrays whose sum equals K; are values negative (decides the method) and can the input be a stream? Grid: building communities as connected components, is adjacency four-way, and what marks a building. The scraped solution covers only the array part.

BRUTE FORCE TO SAY OUT LOUD

Array: all subarrays with running sums, O(n^2). Grid: none needed, a flood fill is the baseline.

OPTIMAL IDEA

Array: prefix sums with a dict of how many times each prefix has occurred; for each element add the count of (prefix - K), then record the prefix. O(n) and it works online for a stream. A sliding window fails with negative values. Grid: DFS/BFS (iterative) or Union-Find to count connected components of building cells.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Array: [1,1,1] with K = 2 gives 2, [1,2,3] with K = 3 gives 2, negatives, K = 0 with zeros, an empty array. Grid: no buildings, one building, diagonal buildings (separate).

LIKELY FOLLOW-UPS

Array: longest subarray with sum K, or counting in a stream. Grid: the largest community, or communities as buildings are added.

PITFALLS

Not seeding the prefix dict with {0: 1}. Using a window technique with negative numbers.

HOW TO THINK ABOUT IT

Subarray sums with negatives allowed cannot use a sliding window; use prefix sums: a subarray ending here sums to K exactly when an earlier prefix equals (current prefix - K), so a dictionary of prefix counts answers it in one pass and works for streams. Communities are connected components, so flood fill or union-find over the building cells.

Basis: Two tasks: count subarrays with sum equal to K (LeetCode 560) and count connected building communities in a grid (LeetCode 200 idea). Assumed rule: buildings are '1' cells, communities connect horizontally and vertically.

BRUTE FORCE · Task 1: try every subarray

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, K):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            if total == K:
                count += 1
    return count
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

O(n^2). A dictionary of prefix sums (how many earlier prefixes equal current - K) is O(n) and also works on a stream and with negative numbers.

BRUTE FORCE · Task 2: relabel cells until the labels stop changing

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE · Task 2: relabel cells until the labels stop changing

```
label = {}
for i, cell in enumerate(buildings):
    label[cell] = i
changed = True
while changed:
    changed = False
    for cell in buildings:
        group = [cell] + neighbours(cell)
        best = min(label[x] for x in group)
        if best < label[cell]:
            label[cell] = best
            changed = True
answer = len(set(label.values()))
```

TIME	SPACE
O((m * n)^2) worst case	O(m * n)

WHY IT IS SLOW

Labels spread one step per round, so a long snake needs about m * n rounds of O(m * n): O((m * n)^2). Union-Find or one flood fill is near-linear.

OPTIMAL SOLUTION · Task 1: prefix sums in a dictionary

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task 2: Union-Find over building cells

TIME	SPACE
O(m * n * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The numbers are all positive.

A sliding window works in O(1) memory: grow the right end, shrink the left while the sum exceeds K. With negatives the window is not monotone, which is why prefix sums are needed.

FOLLOW-UP 2 Longest subarray with sum K.

Store the first index of each prefix sum instead of a count, and take the longest distance to prefix - K.

FOLLOW-UP 3 Buildings are added over time and you report the number of communities after each addition.

Union-Find with a counter: +1 for the new building, -1 for each union with a different neighbouring community (LeetCode 305).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve array-sum and city-community
# problems
#
# This question evaluates array-processing and grid/graph
# connectivity skills, covering counting subarrays with a
# target sum and identifying connected building
# communities in a grid; it belongs to the Coding &
# Algorithms domain and involves streaming/online
# algorithm reasoning as well as grid traversal concepts.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The numbers are all positive.
# 2. Longest subarray with sum K.
# 3. Buildings are added over time and you report the
# number of communities after each addition.
#
# INTERVIEW NOTES
# Subarray sums with negatives allowed cannot use a
# sliding window; use prefix sums: a subarray ending here
# sums to K exactly when an earlier prefix equals (current
# prefix - K), so a dictionary of prefix counts answers it
# in one pass and works for streams. Communities are
# connected components, so flood fill or union-find over
# the building cells.
#
# BRUTE FORCE (say it first): Task 1: try every subarray
# Extend each start with a running sum and count the ends
# where it equals K. Correct and the oracle; quadratic.
# Time O(n^2). Space O(1).


# BRUTE FORCE (say it first): Task 1: try every subarray |
# time O(n^2)
def subarray_sum_brute(nums, k):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            count += total == k
    return count


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))
```

```
# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def count_subarrays(nums, k):
    # prefix sums: a subarray ending here has sum k if
    # some
    # earlier prefix equals (prefix - k)
    seen, prefix, count = {0: 1}, 0, 0
    for x in nums:
        prefix += x
```

```
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count


# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def communities(grid):
    # connected groups of buildings (1) in a grid
    seen, count = set(), 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] != 1 or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            stack = [(r, c)]
            while stack:
                x, y = stack.pop()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < len(grid)):
                        continue
                    if not (0 <= b < len(grid[0])):
                        continue
                    land = grid[a][b] == 1
                    if land and (a, b) not in seen:
                        seen.add((a, b))
                        stack.append((a, b))
    return count


# FOLLOW-UP 1: The numbers are all positive.
# Answer: A sliding window works in O(1) memory: grow the
```

```
# right end, shrink the left while the sum exceeds K. With
# negatives the window is not monotone, which is why
# prefix sums are needed.
def count_positive(nums, k):
    # all numbers positive: a sliding window, O(1) memory
    count = left = total = 0
    for right, x in enumerate(nums):
        total += x
        while total > k and left <= right:
            total -= nums[left]
            left += 1
        if total == k:
            count += 1
    return count


# FOLLOW-UP 2: Longest subarray with sum K.
# Answer: Store the first index of each prefix sum instead
# of a count, and take the longest distance to prefix - K.
def longest_subarray(nums, k):
    # store the FIRST index of each prefix sum
    first, prefix, best = {0: -1}, 0, 0
    for i, x in enumerate(nums):
        prefix += x
        if prefix - k in first:
            best = max(best, i - first[prefix - k])
        first.setdefault(prefix, i)
    return best


# FOLLOW-UP 3: Buildings are added over time and you
# report the number of communities after each addition.
# Answer: Union-Find with a counter: +1 for the new
# building, -1 for each union with a different
# neighbouring community (LeetCode 305).
def communities_online(rows, cols, additions):
    # buildings added one by one: count after each
    # (LeetCode 305)
```

```
    parent, count, out = {}, 0, []

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
```

```
    for r, c in additions:
        if (r, c) not in parent:
            parent[(r, c)] = (r, c)
            count += 1
            for dr, dc in STEPS:
                nb = (r + dr, c + dc)
                if nb in parent:
                    a, b = find((r, c)), find(nb)
                    if a != b:
                        parent[a] = b
                        # two communities became one
                        count -= 1
        out.append(count)
    return out


if __name__ == "__main__":
    assert count_subarrays([1, 1, 1], 2) == 2
    assert count_subarrays([1, -1, 0], 0) == 3
    assert count_subarrays([], 0) == 0
    assert longest_subarray([1, -1, 5, -2, 3], 3) == 4
    assert longest_subarray([1, 2], 9) == 0
    assert count_positive([1, 2, 3, 2, 1], 3) == 3
    grid = [[1, 1, 0], [0, 0, 1], [1, 0, 1]]
    assert communities(grid) == 3
    adds = [(0, 0), (0, 1), (1, 2), (2, 2), (1, 1)]
    counts = communities_online(3, 3, adds)
    assert counts == [1, 1, 2, 2, 1]
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int], k: int) -> int:
```

Sample call

```
solution([1, 1, 1], 2)
```

Test cases

Input	Expected
([1, 1, 1], 2)	2
([1, 2, 3], 3)	2

STUDY PATH: question 14 of 167 (Core 90)

Priority score 5.5 | Hash maps and sets | asked 21 days ago | frequency Low

HARD | 1Point3Acres | onsite-coding | 6 of 7

Maximum-Area Rectangle from Planar Points

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TALK NO LEETCODE FREQUENCY PRIORITY 177/228

ABOUT THIS QUESTION

Topic: Core 90 - Hash maps and sets

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #963 Minimum Area Rectangle II

LeetCode match: LC 963 Minimum Area Free Rectangle (related)

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/max-area-rectangle-from-points

QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

Input: a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

APPROACH

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

HOW TO THINK ABOUT IT

Four points form a rectangle exactly when two of its diagonals share the same midpoint and have the same length. So you never need to pick four points: bucket every pair of points by (sum of the two points, squared distance). Two different pairs in one bucket are the two diagonals of a rectangle, and the area is the cross product of two adjacent sides.

HOW TO THINK ABOUT IT

Basis: LeetCode 963, Minimum Area Free Rectangle, with the objective flipped to maximum area.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(points):
    best = 0
    for p, q, r in permutations(points, 3):
        if dot(q - p, r - p) != 0:
            continue  # no right angle at p
        s = q + r - p  # the fourth corner
        if s in points:
            area = abs(cross(q - p, r - p))
            best = max(best, area)
    return best
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

Three nested loops over the points, O(n^3). Bucketing pairs by their diagonal midpoint and length finds rectangles from O(n^2) pairs.

OPTIMAL SOLUTION · Bucket pairs by diagonal midpoint and length

TIME	SPACE
O(n^2) pairs (worst case O(n^3) if every pair shares a bucket)	O(n^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can you do better than O(n^2) pairs?

Not in general: the answer can depend on all pairs, and even deciding whether any rectangle exists is believed to need about n^2 work. The practical gain is in the buckets: most buckets hold one pair, so the work after bucketing is small. State the worst case honestly (points on a circle put many pairs in few buckets).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Maximum-Area Rectangle from Planar Points
#
# Given 2D points, find the maximum area of a rectangle
# whose four corners are input points; sides need not be
# parallel to the axes. Candidates compare it to LC 963
# (Minimum Area Rectangle II) but maximizing. Interviewers
# ask to improve beyond an O(N^3) approach. Input: a list
```

```
# of 2D coordinate points. Return the maximum area of any
# rectangle whose four vertices are all input points.
# Rectangle sides do not need to be parallel to the
# coordinate axes. Follow-up pressure: start from an
# O(N^3) idea, then optimize toward a better complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can you do better than O(n^2) pairs?
#
# INTERVIEW NOTES
# Four points form a rectangle exactly when two of its
# diagonals share the same midpoint and have the same
# length. So you never need to pick four points: bucket
# every pair of points by (sum of the two points, squared
# distance). Two different pairs in one bucket are the two
# diagonals of a rectangle, and the area is the cross
# product of two adjacent sides.
#
# BRUTE FORCE (say it first): Try every triple with a
# right angle
# Pick a corner p and two neighbours q and r with (q - p)
# . (r - p) = 0, then check that the fourth point q + r -
# p exists. Correct and the first thing to say; cubic is
# too slow for thousands of points. Time O(n^3). Space
# O(n).


from collections import defaultdict
from itertools import combinations


# HELPERS for the brute force
def cross(o, p, q):
    # twice the signed area of triangle o, p, q
    ax, ay = p[0] - o[0], p[1] - o[1]
    bx, by = q[0] - o[0], q[1] - o[1]
    return ax * by - ay * bx
```

```
# BRUTE FORCE (say it first)
def max_rect_area_cubic(points):
    # first idea to say aloud: try every corner triple
    pts = set(map(tuple, points))
    best = 0
    for p in pts:
        for q in pts:
            for r in pts:
                if len({p, q, r}) < 3:
                    continue
                # right angle at p between p->q and p->r
                dot = ((q[0] - p[0]) * (r[0] - p[0])
                       + (q[1] - p[1]) * (r[1] - p[1]))
                # the fourth corner must also exist
                s = (q[0] + r[0] - p[0],
                     q[1] + r[1] - p[1])
                if dot == 0 and s in pts:
                    area = abs(cross(p, q, r))
                    best = max(best, area)
    return best


# HELPERS for the base question
# FOLLOW-UP 1: Can you do better than O(n^2) pairs?
# Answer: Not in general: the answer can depend on all
# pairs, and even deciding whether any rectangle exists is
# believed to need about n^2 work.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def all_rectangle_areas(points):
    # duplicates cannot help, so drop them
    pts = list(set(map(tuple, points)))
    buckets = defaultdict(list)
    # every pair is a possible diagonal
    for p, q in combinations(pts, 2):
        # same centre (sum of corners) and same length
        key = (p[0] + q[0], p[1] + q[1],
               (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
        buckets[key].append((p, q))
    # two diagonals in one bucket form a rectangle
    for pairs in buckets.values():
        for (p, q), (r, s) in combinations(pairs, 2):
            # area = |cross| of two sides leaving p
            yield abs(cross(p, r, s))
```

```


# BASE QUESTION (optimal): Bucket pairs by diagonal
# midpoint and length
# Idea: For every pair (p, q) use the key (p.x + q.x, p.y
# + q.y, squared length). All pairs with the same key are
# diagonals of one circle around the same centre, so any
# two of them form a rectangle.
def max_rect_area(points):
    # best area, or 0 when no rectangle exists
    return max(all_rectangle_areas(points), default=0)


# RELATED VARIANT: smallest area (LeetCode 963)
def min_rect_area(points):
    # LeetCode 963 asks for the smallest instead
    return min(all_rectangle_areas(points), default=0)


if __name__ == "__main__":
    # a tilted square of area 2
    sq = [(0, 0), (1, 1), (0, 2), (-1, 1)]
    assert max_rect_area(sq) == 2
    assert max_rect_area_cubic(sq) == 2
    # axis-aligned 2 x 1 rectangle plus a stray point
    rect = [(0, 0), (2, 0), (0, 1), (2, 1), (5, 5)]
    assert max_rect_area(rect) == 2
    # collinear points make no rectangle
    assert max_rect_area([(0, 0), (1, 1), (2, 2)]) == 0
    # the minimum version picks the small one
    two = rect + [(0, 3), (2, 3)]
    assert min_rect_area(two) == 2
    assert max_rect_area(two) == 6
```

NOTES FROM CANDIDATES

Candidates describe it as LC 963 (Minimum Area Rectangle II) with the objective flipped to maximum area.
Asked as a standalone L4 onsite coding round; in another L4 onsite it was the second problem after a CPU scheduling question, and the interviewer explicitly expected a better-than-cubic solution.
Clarify up front: whether duplicate points can appear, what to return when no rectangle exists (commonly 0), and whether integer or floating-point coordinates are used (area precision matters for rotated rectangles).

PREPARATION

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

STUDY PATH: question 15 of 167 (Core 90)

Priority score 5.42 | Hash maps and sets | asked 41 days ago

HARD | IN APP LC 41 | PracHub | Technical Screen | 7 of 7

Smallest Missing Positive Integer in Linear Time and Constant Extra Space

POSTED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 48 30 DAYS 0 PRIORITY 73/228

ABOUT THIS QUESTION

Topic: Core 90 - Hash maps and sets

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #41 First Missing Positive

LeetCode match: LC 41 First Missing Positive

Posted on PracHub: 2026-09-09 (updated 2026-09-28)

Original: https://prachub.com/coding-questions/smallest-missing-positive-integer-in-linear-time-and-constant-extra-space

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **O(n)** time and **O(1)** auxiliary space, where **n = len(nums)**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **1 <= len(nums) <= 10^5**
- **-2147483648 <= nums[i] <= 2147483647**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **nums = [4, -2, 1, 2]**
- Output: **3**
- Explanation: **1** and **2** are present and **3** is not.

Example 2

- Input: **nums = [5, 6, 100, -3]**
- Output: **1**

QUESTION

- Explanation: **1** does not appear, so it is the answer, however large the other values are.

Example 3

- Input: **nums = [1, 2, 2, 3]**
- Output: **4**
- Explanation: **1**, **2** and **3** all appear (**2** twice), so the smallest missing positive integer is **4**.

APPROACH

Cyclic placement: swap each value v in 1..n into index v-1 until settled, then the first index i where nums[i] != i+1 gives the answer (else n+1). O(n) time, O(1) space.

HOW TO THINK ABOUT IT

The answer is between 1 and n + 1, because n numbers can cover at most n of the positive integers. So the array itself can be used as the 'seen' table: put each value v (1 <= v <= n) at index v - 1. After that, the first index i where nums[i] != i + 1 gives the answer i + 1.

Basis: LeetCode 41, First Missing Positive.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    seen = set(nums)
    x = 1
    while x in seen:
        x += 1
    return x
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but O(n) extra memory, which is what the follow-up asks to remove. Placing each value v at index v - 1 inside the array itself, then finding the first mismatch, uses O(1) space.

OPTIMAL SOLUTION · Cyclic placement: put each value v at index v - 1

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without swapping (sign marking).

Replace out-of-range values by n + 1, then for every value v in 1..n make the entry at index v - 1 negative as a 'seen' flag; the first positive entry marks the missing number. Same O(n) time and O(1) space.

FOLLOW-UP 2 The array must not be modified.

Then O(1) space with O(n) time is not possible in general; copy the array (O(n) space) or accept O(n log n) time with a sort of a copy. State the trade-off.

FOLLOW-UP 3 Smallest missing non-negative integer, or the k-th missing positive.

Shift the target range by one for non-negative. For the k-th missing positive in a sorted array use binary search on 'how many numbers are missing before index i' = a[i] - (i + 1).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Smallest Missing Positive Integer in
# Linear Time and Constant Extra Space
#
# You are given an unsorted list of integers nums, which
# may contain negative numbers, zeros, duplicates, and
# values far larger than the length of the list. Return
# the smallest positive integer, that is the smallest
# integer that is at least 1, that does not appear in
# nums. In the interview, a first solution that ran in
# linear time but used extra memory proportional to the
# input was accepted as a starting point, and the
# interviewer then asked for the same result using only
# constant extra space. Aim for that tighter target. ###
# Function Signature python def
# first_missing_positive(nums: list[int]) -> int: ###
# Rules - Target complexity: O(n) time and O(1) auxiliary
# space, where n = len(nums). The list may be rearranged
# or overwritten in place; its contents after the call do
# not matter. - Return only the missing value.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without swapping (sign marking).
# 2. The array must not be modified.
# 3. Smallest missing non-negative integer, or the k-th
# missing positive.
#
# INTERVIEW NOTES
# The answer is between 1 and n + 1, because n numbers can
# cover at most n of the positive integers. So the array
# itself can be used as the 'seen' table: put each value v
# (1 <= v <= n) at index v - 1. After that, the first
# index i where nums[i] != i + 1 gives the answer i + 1.
#
# BRUTE FORCE (say it first): Hash set of the values, then
# count up
# Put all numbers in a set and test 1, 2, 3, ... until one
# is missing. Linear time but linear extra memory, which
# is the version the interviewer wants to improve. Time
# O(n). Space O(n).
```

```
from bisect import bisect_left


# BASE QUESTION (optimal): Cyclic placement: put each
```

```
# value v at index v - 1
# Idea: For each position, while nums[i] is in 1..n and
# not already at its home index, swap it to index nums[i]
# - 1. Each swap puts one value in its final place, so the
# total number of swaps is at most n.
def first_missing_positive(nums):
    # put each value v in 1..n at index v - 1 by swapping
    a, n = list(nums), len(nums)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            j = a[i] - 1
            a[i], a[j] = a[j], a[i]
    # the first index that does not hold index + 1
    for i in range(n):
        if a[i] != i + 1:
            return i + 1
    return n + 1


# EASY VERSION: O(n) memory (say it first)
def first_missing_set(nums):
    # the easy O(n) memory answer (fine as a first
    # version)
    seen, v = set(nums), 1
    while v in seen:
        v += 1
    return v


# VARIANT: k-th missing positive
def kth_missing_positive(sorted_nums, k):
    # follow-up: k-th missing positive in a sorted array.
    # missing before index i is a[i] - (i + 1): binary
    # search
    lo, hi = 0, len(sorted_nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if sorted_nums[mid] - (mid + 1) < k:
            lo = mid + 1
        else:
            hi = mid
    return lo + k


# FOLLOW-UP 1: Do it without swapping (sign marking).
```

```
# Answer: Replace out-of-range values by n + 1, then for
# every value v in 1..n make the entry at index v - 1
# negative as a 'seen' flag; the first positive entry
# marks the missing number. Same O(n) time and O(1) space.
def first_missing_marking(nums):
    # no swaps: use the sign of a[v - 1] as a "seen" flag
    a, n = list(nums), len(nums)
    # values outside 1..n cannot matter: replace them
    for i in range(n):
        if a[i] <= 0 or a[i] > n:
            a[i] = n + 1
    for i in range(n):
        v = abs(a[i])
        if v <= n:
            a[v - 1] = -abs(a[v - 1])
    # the first positive entry marks the missing number
    for i in range(n):
        if a[i] > 0:
            return i + 1
    return n + 1
```

```
if __name__ == "__main__":
    cases = [([3, 4, -1, 1], 2), ([1, 2, 0], 3),
             ([7, 8, 9, 11, 12], 1), ([], 1),
             ([1, 1, 1], 2), ([2, 1], 3)]
    for nums, want in cases:
        assert first_missing_positive(nums) == want
        assert first_missing_marking(nums) == want
        assert first_missing_set(nums) == want
    # the input list is left untouched
    original = [3, 4, -1, 1]
    first_missing_positive(original)
    assert original == [3, 4, -1, 1]
    # 2, 3, 4, 7, 11: the 5th missing positive is 9
    assert kth_missing_positive([2, 3, 4, 7, 11], 5) == 9
    assert kth_missing_positive([1, 2, 3, 4], 2) == 6
```

FUNCTION AND TEST CASES

Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Sample call

```
first_missing_positive([4, -2, 1, 2])
```

Test cases

Input	Expected
([4, -2, 1, 2],)	3
([5, 6, 100, -3],)	1